Виды установок Kubernetes. Kubespray

Занятие 02 · День 1, после обеда

Ставим кластер на учебный стенд — вместе и по шагам

Способы получить кластер

Способ	Для чего	Сложность
kind, minikube, k3d	разработка на ноутбуке, CI-тесты	минимальная
k3s, k0s, MicroK8s	edge, небольшие кластеры, лабы	низкая
kubeadm	«ванильный» кластер руками, основа CKA	средняя
Kubespray	production на своих серверах, Ansible	средняя
RKE2, OpenShift, Deckhouse	платформы с поддержкой вендора	средняя–высокая
Managed (EKS, GKE, AKS, Yandex)	облако, control plane не ваш	низкая
Cluster API, Talos	кластеры как код, массово	высокая

Как выбрать

- **Где** будет работать: облако / свои серверы / ноутбук / без интернета
- **Кто** будет сопровождать: есть ли команда, нужна ли поддержка вендора
- **Повторяемость**: сколько кластеров, как часто пересоздаём
- **Обновления**: насколько просто перейти на новую версию
- **Требования ИБ**: сертификация, изолированный контур, свой registry

> Для своих серверов без облака и без вендора Kubespray — один из самых распространённых вариантов.

kubeadm — фундамент

```
# на master
kubeadm init --pod-network-cidr=10.244.0.0/16
# на worker
kubeadm join 192.168.56.11:6443 --token ... --discovery-token-ca-cert-hash sha256:...
```

kubeadm **не делает**:

- не готовит ОС (swap, модули ядра, sysctl, runtime)
- не ставит CNI, ingress, storage
- не автоматизирует множество узлов

Kubespray = Ansible поверх kubeadm + подготовка ОС + аддоны + жизненный цикл.

Kubespray

- Проект **Kubernetes SIGs** — `github.com/kubernetes-sigs/kubespray`
- Набор Ansible-ролей и плейбуков: `cluster.yml`, `scale.yml`, `upgrade-cluster.yml`, `remove-node.yml`, `reset.yml`
- ОС: Ubuntu 22.04/24.04/26.04, Debian, RHEL/Alma/Rocky, Fedora, Flatcar…
- HA control plane, внешний/stacked etcd, offline-установка

Версии в Kubespray v2.32.0 (сентябрь 2026)

Kubernetes	etcd	containerd	Calico	CoreDNS	Helm
1.36.4	3.6.14	2.3.5	3.31.7	1.14.2	3.22.0

Что изменилось в последних версиях

- **v2.31**: аддон **ingress-nginx удалён** — проект ingress-nginx прекращён (retired). Для входящего трафика — **Gateway API** (переменная `gateway_api_enabled`) + реализация (Envoy Gateway, Cilium, ...)
- **v2.31**: удалён аддон Kubernetes Dashboard; cgroup v1 не поддерживается по умолчанию
- **v2.32**: нужен **ansible-core 2.19** (пакет `ansible==12.3.0`) — условия в `when:` должны быть строго булевыми
- Валидация: плейбук остановится, если в inventory остались **удалённые переменные**

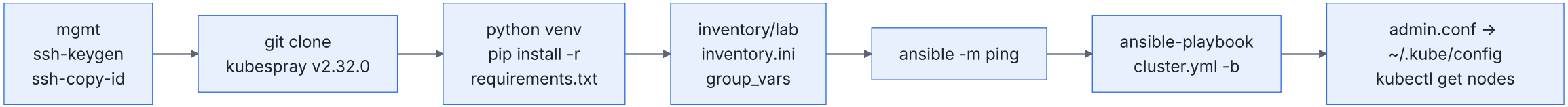
> Всегда читайте Release Notes перед обновлением Kubespray.

Ansible за 5 минут

Понятие	Что это	Пример
Inventory	список хостов и групп	`[kube_node]` → `worker-1`, `worker-2`
Module	единица действия	`apt`, `copy`, `systemd`, `shell`
Task	вызов модуля с параметрами	«установить containerd»
Role	набор задач, шаблонов, переменных	`roles/etcd`
Playbook	какие роли на какие группы	`cluster.yml`
group_vars	переменные для групп	`group_vars/k8s_cluster/*.yml`

- **Agentless**: только SSH + Python на узлах
- **Идемпотентность**: повторный запуск не ломает, а доводит до нужного состояния
- `become: true` / `-b` — выполнять через `sudo`

План установки



Всё выполняется **на машине mgmt**. На узлы кластера руками не заходим.

Требования

Машина с Ansible (mgmt):

- Python **3.11+** (в Ubuntu 24.04 — 3.12), `python3-venv`, `git`
- Ansible из `requirements.txt` Kubespray — ставим в **virtualenv**

Узлы кластера:

- Доступ по **SSH по ключу**, пользователь с **sudo** без пароля (в Vagrant — `vagrant`)
- Доступ в интернет (образы, бинарники) или настроенный offline-режим
- Master ≥ 2 CPU / 1.5 GB RAM, worker ≥ 1 GB RAM (для GitLab — больше!)
- Firewall выключен или открыты порты (6443, 2379-2380, 10250, Calico...)

Установка по шагам

Показываю на своём стенде — вы повторяете на практике

ДЕМО Шаг 0. Стенд

```
# Linux / macOS
export VAGRANT_DEFAULT_PROVIDER=virtualbox   # libvirt | qemu
# Windows PowerShell:  $env:VAGRANT_DEFAULT_PROVIDER = "virtualbox"

cd vagrant
vagrant up
vagrant status
vagrant ssh mgmt
```

```
vagrant@mgmt:~$ cat /etc/hosts | grep -A6 devops-lab
vagrant@mgmt:~$ ping -c1 master-1
```

ДЕМО Шаг 1. SSH-ключ и доступ к узлам

```
ssh-keygen -t ed25519 -C "mgmt@devops-lab"
#   Enter file: (Enter — по умолчанию ~/.ssh/id_ed25519)

for ip in 192.168.56.11 192.168.56.21 192.168.56.22; do
  ssh-copy-id -o StrictHostKeyChecking=accept-new vagrant@$ip   # пароль: vagrant
done

# проверяем: вход без пароля и sudo без пароля
for ip in 192.168.56.11 192.168.56.21 192.168.56.22; do
  ssh vagrant@$ip 'hostname; sudo -n true && echo sudo-ok'
done
```

- **ed25519** — короче и надёжнее RSA; приватный ключ никогда не покидает mgmt
- `ssh-copy-id` дописывает **публичный** ключ в `~/.ssh/authorized_keys` узла

ДЕМО Шаг 2. Kubespray и Ansible

```
git clone --branch v2.32.0 --depth 1 \
  https://github.com/kubernetes-sigs/kubespray.git ~/kubespray
cd ~/kubespray

python3 -m venv ~/kubespray-venv
source ~/kubespray-venv/bin/activate       # (kubespray-venv) в приглашении
pip install -U pip
pip install -r requirements.txt

ansible --version                          # ansible [core 2.19.x]
```

- **Фиксируем версию** (тег) — `master` меняется каждый день
- **venv** — своя версия Ansible для каждой версии Kubespray, системный Python не трогаем
- Новый терминал → снова `source ~/kubespray-venv/bin/activate`

ДЕМО Шаг 3. Inventory

```
cp -rfp inventory/sample inventory/lab
vim inventory/lab/inventory.ini
```

```
[all:vars]
ansible_user=vagrant

[kube_control_plane]
master-1 ansible_host=192.168.56.11 ip=192.168.56.11 etcd_member_name=etcd1

[etcd:children]
kube_control_plane

[kube_node]
worker-1 ansible_host=192.168.56.21 ip=192.168.56.21
worker-2 ansible_host=192.168.56.22 ip=192.168.56.22
```

Зачем `ip=`, если есть `ansible_host=`?

```
vagrant@worker-1:~$ ip -br a
lo        UNKNOWN  127.0.0.1/8
eth0      UP       10.0.2.15/24      ← NAT VirtualBox — ОДИНАКОВЫЙ на всех ВМ!
eth1      UP       192.168.56.21/24  ← сеть стенда
```

- Без `ip=` kubelet и etcd возьмут адрес **интерфейса маршрута по умолчанию** — `10.0.2.15`
- Итог: все узлы с одинаковым InternalIP, поды на разных узлах не видят друг друга
- `ansible_host` — куда ходит **Ansible**; `ip` — где слушает **Kubernetes**
- Для Calico отдельно: `calico_ip_auto_method: "cidr=192.168.56.0/24"`

> Классическая ошибка Vagrant-стендов. В облаке с одним интерфейсом `ip=` можно не указывать.

Шаг 4. Структура group_vars

```
inventory/lab/
├── inventory.ini
└── group_vars/
    ├── all/
    │   ├── all.yml            # общие: прокси, DNS, загрузки
    │   ├── containerd.yml     # настройки runtime, registry mirrors
    │   └── etcd.yml
    └── k8s_cluster/
        ├── k8s-cluster.yml    # версия, сеть, kube-proxy, kubeconfig
        ├── k8s-net-calico.yml # параметры Calico
        └── addons.yml         # metrics-server, local-path, MetalLB, cert-manager…
```

Значения по умолчанию — в `roles/kubespray_defaults/defaults/main/`. В inventory меняем **только нужное**.

> В файлах-примерах многие строки **закомментированы**: `# kube_proxy_strict_arp: false`. Меняя значение, не забудьте убрать `#`.

ДЕМО Шаг 5. Ключевые переменные

`group_vars/k8s_cluster/k8s-cluster.yml`

```
kube_network_plugin: calico             # CNI (по умолчанию)
kube_proxy_mode: ipvs                   # ipvs | iptables | nftables
kube_proxy_strict_arp: true             # ← нужно MetalLB (L2) при ipvs
kube_service_addresses: 10.233.0.0/18
kube_pods_subnet: 10.233.64.0/18
cluster_name: cluster.local
```

`group_vars/k8s_cluster/k8s-net-calico.yml`

```
calico_ip_auto_method: "cidr=192.168.56.0/24"
```

Версия Kubernetes (`kube_version`) не задана → берётся последняя поддерживаемая в v2.32 — **1.36.x**

ДЕМО Шаг 5. Аддоны

`group_vars/k8s_cluster/addons.yml`

```yaml
metrics_server_enabled: true            # kubectl top
local_path_provisioner_enabled: true    # StorageClass по умолчанию — для GitLab
metallb_enabled: true                   # Service type: LoadBalancer
metallb_speaker_enabled: true
metallb_config:
  address_pools:
    primary:                        # для обычных сервисов
      ip_range:
        - 192.168.56.201-192.168.56.220
      auto_assign: true
    gitlab:                         # .200 бережём для GitLab (выдаётся только по запросу)
      ip_range:
        - 192.168.56.200-192.168.56.200
      auto_assign: false
  layer2:
    - primary
    - gitlab
```

MetalLB в режиме **L2** отвечает на ARP-запросы за «внешний» IP — как будто это ещё один хост в сети `192.168.56.0/24`.

ДЕМО Шаг 5. Заглядываем в будущее: registry GitLab

На 4-й день GitLab будет раздавать образы по **HTTP** (учебный стенд без TLS). Скажем containerd об этом сразу, чтобы потом не перезапускать установку:

`group_vars/all/containerd.yml`

```
containerd_registries_mirrors:
  - prefix: registry.192.168.56.200.nip.io
    server: http://registry.192.168.56.200.nip.io
    mirrors:
      - host: http://registry.192.168.56.200.nip.io
        capabilities: ["pull", "resolve"]
        skip_verify: true
```

- Kubespray создаст `/etc/containerd/certs.d/registry.../hosts.toml` на каждом узле
- `*.nip.io` — публичный DNS: `x.192.168.56.200.nip.io` → `192.168.56.200`

ДЕМО Шаг 6. Проверка доступа

```
cd ~/kubespray
ansible -i inventory/lab/inventory.ini all -m ping
```

```
master-1 | SUCCESS => { "ping": "pong" }
worker-1 | SUCCESS => { "ping": "pong" }
worker-2 | SUCCESS => { "ping": "pong" }
```

```
# ad-hoc команды — удобно для диагностики
ansible -i inventory/lab/inventory.ini all -b -m shell -a 'free -m | head -2'
ansible -i inventory/lab/inventory.ini kube_node -m setup -a 'filter=ansible_default_ipv4'
```

ДЕМО Шаг 7. Установка

```
ansible-playbook -i inventory/lab/inventory.ini -b cluster.yml
```

Что происходит (15–30 минут):

1. Проверки: версии, ОС, переменные, ресурсы
2. Подготовка ОС: swap off, модули `br_netfilter` / `overlay`, sysctl, пакеты
3. Скачивание бинарников и образов, установка **containerd**
4. **etcd** (systemd-сервис), сертификаты
5. **kubeadm init** на master, **kubeadm join** на workers
6. **Calico**, **CoreDNS**, nodelocaldns, аддоны (metrics-server, local-path, MetalLB)

Полезно: `-v` (подробнее), `--tags` / `--skip-tags`, `--limit worker-2`.

ДЕМО Шаг 8. kubectl на master-1

Kubespray уже поставил `kubectl` на master-1. Даём к нему доступ пользователю `vagrant`:

```
# на хосте: vagrant ssh master-1   (или с mgmt: ssh master-1)
mkdir -p ~/.kube
sudo cp /etc/kubernetes/admin.conf ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
kubectl get nodes

echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl; complete -o default -F __start_kubectl k' >> ~/.bashrc
source ~/.bashrc
```

`admin.conf` — это **полный доступ** к кластеру (cluster-admin). Не коммитьте его в Git и не пересылайте в мессенджерах.

ДЕМО Проверка кластера

```
kubectl get nodes -o wide          # INTERNAL-IP = 192.168.56.x !
kubectl get pods -A
kubectl top nodes                  # metrics-server
kubectl get storageclass           # local-path (default)
kubectl -n metallb-system get ipaddresspool,l2advertisement

# проверим LoadBalancer
kubectl create deployment hello --image=nginx:1.28-alpine
kubectl expose deployment hello --port=80 --type=LoadBalancer
kubectl get svc hello              # EXTERNAL-IP 192.168.56.201
curl -s http://192.168.56.201 | head -4
kubectl delete svc,deployment hello
```

Жизненный цикл кластера

Задача	Плейбук
Добавить worker	добавить в inventory → `ansible-playbook scale.yml --limit=worker-3`
Удалить узел	`ansible-playbook remove-node.yml -e node=worker-2`
Обновить Kubernetes	новая версия Kubespray / `kube_version` → `upgrade-cluster.yml` (по одной минорной версии!)
Снести кластер	`ansible-playbook reset.yml`

Совет для стенда: после успешной установки сделайте снимок — `vagrant snapshot save cluster-ok` (на qemu — `./snapshot-qemu.sh save cluster-ok`). Откат за минуту вместо переустановки за полчаса.

Частые проблемы

Симптом	Причина / решение
`UNREACHABLE` в ping	ключ не скопирован, неверный `ansible_user`
ошибка про версию Ansible	не активирован venv / не та версия `requirements.txt`
Узлы с IP `10.0.2.15`	не указан `ip=` в inventory
Поды Calico `CrashLoopBackOff`	неверный `calico_ip_auto_method`
Таймауты скачивания	нет интернета / DNS на узлах; перезапустить плейбук — он идемпотентен
OOM, зависание ВМ	мало RAM: уменьшите нагрузку на хост, проверьте `LAB_*_MEM`
`EXTERNAL-IP <pending>`	MetalLB не включён или нет `kube_proxy_strict_arp`

Итоги

- Способов получить кластер много — выбор зависит от среды и команды
- **Kubespray** = Ansible + kubeadm + подготовка ОС + аддоны + жизненный цикл
- Всё начинается с **SSH-ключей** и правильного **inventory**
- В Vagrant обязательно `ip=` и автоопределение IP для Calico
- Изменения — через **group_vars** в своём `inventory/lab`, версии фиксируем тегом

Практика 02

Устанавливаем свой кластер Kubespray

Задание — на платформе testlms.kz